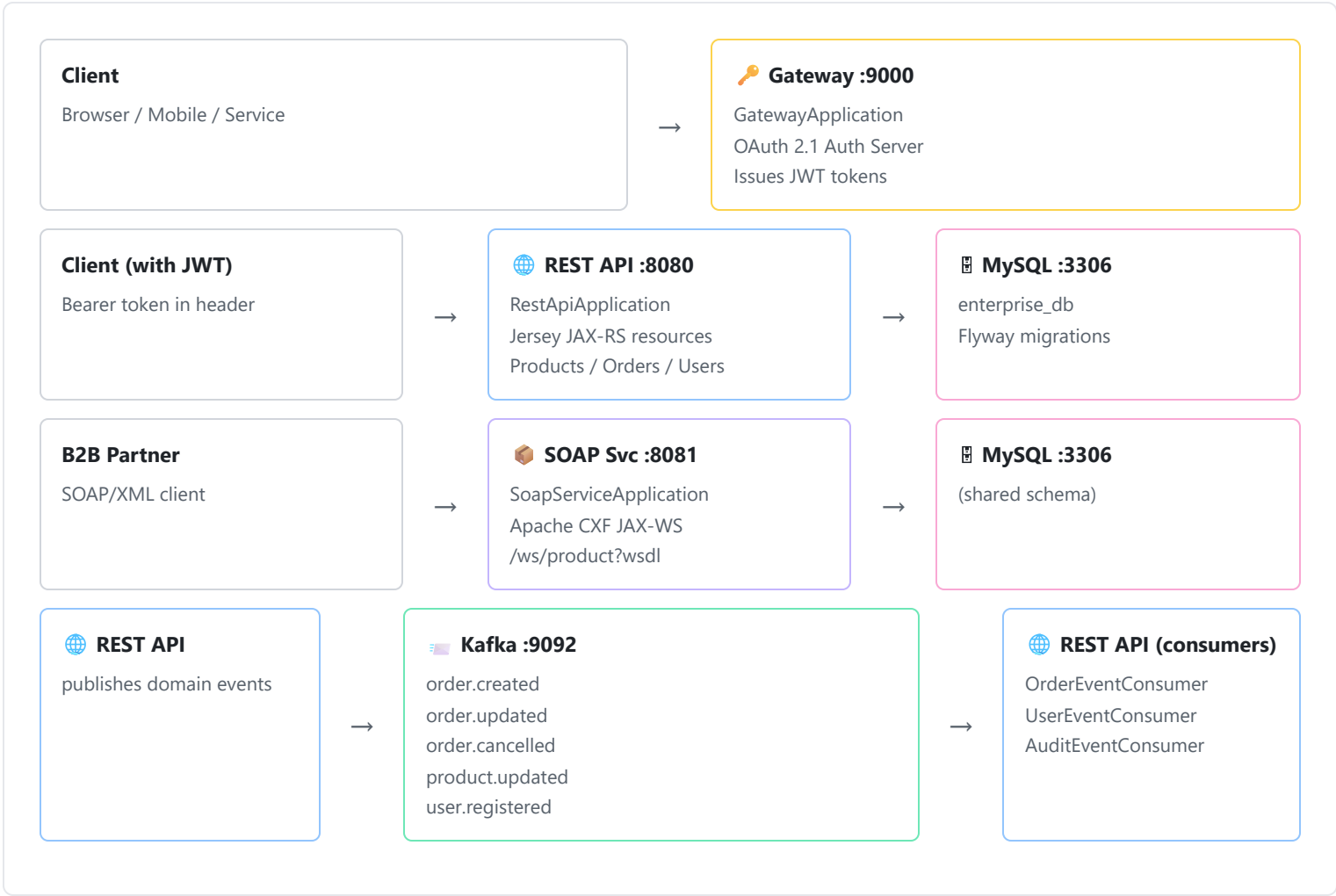
Enterprise Platform — Execution Flow

Spring Boot 3.2 · JAX-RS/Jersey · Apache CXF · Kafka · OAuth2 · MySQL · Maven multi-module

1. System Architecture at a Glance



2. Module Dependency Map

Module	Role	Depends On	Runnable?
`enterprise-common`	Shared constants, `ApiResponse<T>`, `PagedResponse<T>`, exceptions	—	No (library)
`enterprise-domain`	JPA entities (`User`, `Product`, `Order`), DTOs, MapStruct mappers	common	No (library)
`enterprise-repository`	Spring Data JPA repositories with JPQL queries	domain	No (library)
`enterprise-kafka`	Kafka producer, 3 consumers, 5 event classes	common	No (library)
`enterprise-security`	OAuth2 Auth Server config + Resource Server config	—	No (library)
`enterprise-rest-api`	JAX-RS REST endpoints + business services	all above	**Yes (:8080)**
`enterprise-soap-service`	Apache CXF SOAP endpoint	domain, repository	**Yes (:8081)**
`enterprise-gateway`	OAuth 2.1 Authorization Server	security	**Yes (:9000)**

---

3. Startup Sequence

Step A — Build (all 8 modules in order)

1

Maven resolves the dependency graph

Parent `pom.xml` declares all 8 modules. Maven compiles them bottom-up: `common → domain → repository → kafka → security → rest-api / soap-service / gateway`.

Lombok and MapStruct annotation processors run at compile time, generating boilerplate (`@Builder`, `@Data`) and mapper implementations.

2

Flyway migrations run on first REST API or SOAP startup

When `enterprise-rest-api` starts (local/docker profile), Flyway runs `V1__initial_schema.sql` (7 tables, FK constraints, indexes) and `V2__seed_data.sql` (categories, admin user, sample products).

The SOAP service has `flyway.enabled=false` — it shares the schema but doesn't own migrations.

Step B — Service startup order (Docker Compose)

1. **Zookeeper** starts (Kafka coordinator)
2. **Kafka** starts, depends on Zookeeper being healthy
3. **MySQL** starts (health-checked with `mysqladmin ping`)
4. **Kafka UI** starts, depends on Kafka
5. **Gateway (:9000)** depends on MySQL healthy — generates RSA-2048 key pair at boot
6. **REST API (:8080)** depends on MySQL healthy + Kafka healthy — runs Flyway, wires Jersey, Kafka, Security
7. **SOAP Service (:8081)** depends on MySQL healthy — publishes CXF WSDL

---

4. REST API Request Lifecycle (e.g. POST /api/v1/orders)

1

Security **Spring Security Filter Chain**

Every HTTP request first passes through `ResourceServerConfig.resourceServerFilterChain()`. The filter checks the `Authorization: Bearer <token>` header.

If the path is `/api/v1/**` and the token is missing or invalid → **401 Unauthorized** returned immediately. The JWKS public key is fetched once from `localhost:9000/oauth2/jwks` to validate JWT signatures.

2

Filter **CorrelationIdFilter (Decorator)**

After security passes, Jersey runs `CorrelationIdFilter.filter()` at `@Priority(1)`. It reads or generates a `X-Correlation-Id` UUID and puts it in the SLF4J MDC.

Every log line for this request now carries the correlation ID. On response, the same ID is echoed back as a response header and cleared from MDC.

Resource **JAX-RS Resource (e.g. OrderResource)**

Jersey routes the request to the matching `@Path` / `@POST` method in `OrderResource`. The `@PreAuthorize("hasAuthority('SCOPE_write')")` annotation is enforced by Spring Method Security before the method body runs.

The JSON body is deserialized into `OrderDto`. Bean Validation (`@Valid`) triggers Jakarta Validation — any constraint violations immediately throw a `ConstraintViolationException`.

4

Service

OrderService (Facade)

`OrderService.createOrder()` orchestrates all business logic in one transaction (`@Transactional`):

a) Lookup `User` by userId → `ResourceNotFoundException` if not found
b) For each `OrderItemDto`: lookup `Product`, check `isAvailableForPurchase(qty)`, call `product.deductStock(qty)`, save Product
c) Build `Order` aggregate, call `order.addItem()` — which auto-calls `recalculateTotal()`
d) Persist the Order via `orderRepository.save()`
e) Publish `OrderCreatedEvent` to Kafka

5

DB

JPA / Hibernate → MySQL

Hibernate executes the batched INSERT statements (batch size 25). `@Version` on Order provides optimistic locking — concurrent updates throw `OptimisticLockException`. HikariCP manages the connection pool.

6

Kafka

KafkaEventProducer → order.created topic

`KafkaEventProducer.publish(TOPIC_ORDER_CREATED, orderNumber, event)` sends the `OrderCreatedEvent` asynchronously. The producer is configured with `acks=all`, `retries=3`, and idempotence enabled for at-least-once delivery.

The future callback logs success (partition + offset) or failure. A send failure does **not** roll back the DB transaction — by design, events are best-effort fire-and-forget.

7

Response

ApiResponse wrapper → HTTP 201

`OrderMapper.toDto()` converts the saved entity to an `OrderDto`. The resource returns `Response.created(location).entity(ApiResponse.success(dto, "...")).build()`.

The `ApiResponse<T>` envelope always carries: `success` flag, ISO-8601 `timestamp`, `data` payload, and optional `message`. The Location header points to `/api/v1/orders/{id}`.

8

Kafka Consumer

OrderEventConsumer (async, separate thread)

In a concurrent consumer thread (concurrency=3), `OrderEventConsumer.handleOrderCreated()` receives the event from the `order.created` topic. It logs a confirmation email simulation and calls `ack.acknowledge()` (manual ACK mode).

If processing throws, the method deliberately skips `ack` so Kafka retries the message on the next poll. `AuditEventConsumer` also processes the same event in a separate consumer group for audit logging.

---

5. Exception Handling — Chain of Responsibility

Exception	HTTP Status	Example Trigger
`ResourceNotFoundException`	404 Not Found	Order ID not found in DB
`BusinessValidationException`	409 Conflict	Product out of stock, duplicate SKU
`UnauthorizedException`	401 Unauthorized	Explicit auth check failure
`ConstraintViolationException`	400 Bad Request	Missing required field, invalid email
Anything else	500 Internal Server Error	Unexpected runtime exception

All responses use the same `ApiResponse.error(message)` envelope. `GlobalExceptionMapper` catches every exception from every JAX-RS resource.

---

6. OAuth2 Token Flow

Client Credentials (service-to-service)

1. Service sends `POST /oauth2/token` to Gateway with `grant_type=client_credentials` + Basic Auth (`enterprise-service-client:service-secret`)
2. Gateway validates client, generates RS256 JWT signed with the runtime RSA-2048 key, returns token (valid 30 min)
3. Service attaches `Authorization: Bearer <token>` to REST API requests
4. REST API's `ResourceServerConfig` fetches `GET /oauth2/jwks` from Gateway (once, cached), verifies token signature and expiry

5. Spring Security populates `SecurityContext` with scopes from JWT claims
6. `@PreAuthorize("hasAuthority('SCOPE_write')")` allows or denies the request

Authorization Code + PKCE (browser client)

1. Browser redirects user to `GET /oauth2/authorize?response_type=code&code_challenge=...&client_id=enterprise-web-client`
2. Gateway shows login form (Spring's default form login)
3. User logs in → Gateway issues authorization code, redirects to `http://localhost:3000/callback?code=...`
4. Frontend exchanges code + PKCE verifier for JWT access token (1 hour) + refresh token (7 days)
5. Subsequent API calls carry the JWT — validated same as above

---

7. SOAP Service Request Flow

1

CXF **SOAP XML arrives at `POST /ws/product`**

Apache CXF deserializes the XML envelope into a Java method call on `ProductWebServiceImpl`. The WSDL is auto-generated and served at `/ws/product?wsdl`.

2

Adapter **ProductWebServiceImpl delegates to ProductRepository**

The implementation adapts the JAX-WS SOAP contract to the same Spring Data `ProductRepository` used by the REST API — no business logic duplication.

3

DB **Query executes against shared MySQL schema**

Returns a SOAP-safe XML DTO. Apache CXF serializes the response back to XML and returns HTTP 200 with a SOAP envelope.

---

8. Kafka Event Map

Topic (AppConstants)	Event Class	Published By	Consumers (Group)
`order.created`	`OrderCreatedEvent`	`OrderService`	OrderEventConsumer (notification), AuditEventConsumer (audit)
`order.updated`	`OrderUpdatedEvent`	`OrderService`	OrderEventConsumer (notification)
`order.cancelled`	`OrderCancelledEvent`	`OrderService`	OrderEventConsumer (notification)
`product.updated`	`ProductUpdatedEvent`	`ProductService`	AuditEventConsumer (audit)
`user.registered`	`UserRegisteredEvent`	`UserService`	UserEventConsumer (notification), AuditEventConsumer (audit)

> All events extend `DomainEvent` and carry: `eventId` (UUID), `eventType`, `occurredAt`, `sourceService`. Producer uses `acks=all` + idempotence. Consumers use manual ACK mode — a failed consumer simply skips `ack` to trigger retry.

---

9. Spring Profiles & How Configuration Merges

Base `application.yml` always loads first (port, JPA batch settings, Flyway locations, log pattern), then the active profile overrides only the differing values:

Setting	h2	local	docker
`Database`	H2 in-memory	MySQL localhost:3306	MySQL container (mysql:3306)
`Kafka broker`	Disabled (`spring.kafka.enabled=false`)	localhost:9092	kafka:29092 (container)
`OAuth2 JWKS`	AutoConfig excluded	localhost:9000	gateway:9000 (container)
`Flyway`	Disabled (ddl-auto=create-drop)	Enabled	Enabled
`show-sql`	true	true	false
`Log level`	DEBUG	DEBUG	INFO

Setting	h2	local	docker
`HikariCP pool`	default	max=5	max=20

> **KafkaConfig guard:** The `@ConditionalOnProperty(name="spring.kafka.enabled", havingValue="true", matchIfMissing=true)` annotation on `KafkaConfig` means the entire Kafka bean graph is skipped when the `h2` profile is active. The app boots with zero external dependencies.

---

10. How to Run

Mode	Commands	What Starts
H2 *zero deps*	`mvn clean install -DskipTests` `mvn spring-boot:run -pl enterprise-rest-api -Dspring-boot.run.profiles=h2`	REST API only · H2 console at `:8080/h2-console` · No Gateway or Kafka needed
Local *MySQL + Kafka*	Start MySQL + Kafka locally, then: `mvn spring-boot:run -pl enterprise-gateway -Dspring-boot.run.profiles=local` `mvn spring-boot:run -pl enterprise-rest-api -Dspring-boot.run.profiles=local`	Gateway + REST API (+ optional SOAP) against local MySQL
Docker *full stack*	`mvn clean package -DskipTests` `docker compose up --build`	All 7 containers · Profiles set automatically via `SPRING_PROFILES_ACTIVE=docker`
